Stream JSON Lines

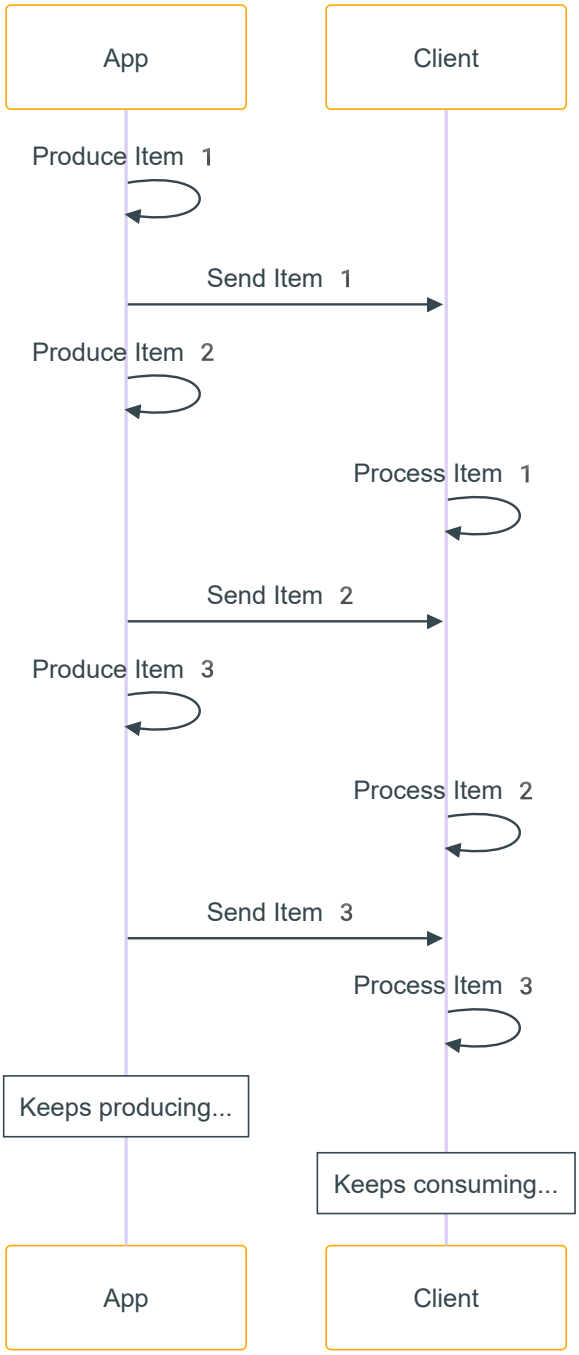
You could have a sequence of data that you would like to send in a "**stream**", you could do it with **JSON Lines**.

> **Info**
>
> Added in FastAPI 0.134.0.

What is a Stream?

"**Streaming**" data means that your app will start sending data items to the client without waiting for the entire sequence of items to be ready.

So, it will send the first item, the client will receive and start processing it, and you might still be producing the next item.



It could even be an infinite stream, where you keep sending data.

JSON Lines

In these cases, it's common to send "**JSON Lines**", which is a format where you send one JSON object per line.

A response would have a content type of `application/jsonl` (instead of `application/json`) and the body would be something like:

```
{"name": "Plumbus", "description": "A multi-purpose household device."}
{"name": "Portal Gun", "description": "A portal opening device."}
{"name": "Meeseeks Box", "description": "A box that summons a Meeseeks."}
```

It's very similar to a JSON array (equivalent of a Python list), but instead of being wrapped in `[]` and having `,` between the items, it has **one JSON object per line**, they are separated by a new line character.

> **Info**
>
> The important point is that your app will be able to produce each line in turn, while the client consumes the previous lines.

> **Technical Details**
>
> Because each JSON object will be separated by a new line, they can't contain literal new line characters in their content, but they can contain escaped new lines (`\n`), which is part of the JSON standard.
>
> But normally you won't have to worry about it, it's done automatically, continue reading. 🤓

Use Cases

You could use this to stream data from an **AI LLM** service, from **logs** or **telemetry**, or from other types of data that can be structured in **JSON** items.

> **Tip**
>
> If you want to stream binary data, for example video or audio, check the advanced guide: Stream Data.

Stream JSON Lines with FastAPI

To stream JSON Lines with FastAPI you can, instead of using `return` in your *path operation function*, use `yield` to produce each item in turn.

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async")
def stream_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation")
async def stream_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation")
def stream_items_no_async_no_annotation():
    for item in items:
        yield item
```

If each JSON item you want to send back is of type `Item` (a Pydantic model) and it's an async function, you can declare the return type as `AsyncIterable[Item]`:

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
```

```python
]


@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async")
def stream_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation")
async def stream_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation")
def stream_items_no_async_no_annotation():
    for item in items:
        yield item
```

If you declare the return type, FastAPI will use it to **validate** the data, **document** it in OpenAPI, **filter** it, and **serialize** it using Pydantic.

Tip

As Pydantic will serialize it in the **Rust** side, you will get much higher **performance** than if you don't declare a return type.

Non-async *path operation functions*

You can also use regular `def` functions (without `async`), and use `yield` the same way.

FastAPI will make sure it's run correctly so that it doesn't block the event loop.

As in this case the function is not async, the right return type would be `Iterable[Item]`:

Python 3.10+

```python
# Code above omitted 👆

@app.get("/items/stream-no-async")
def stream_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async")
def stream_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation")
async def stream_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation")
def stream_items_no_async_no_annotation():
    for item in items:
        yield item
```

No Return Type

You can also omit the return type. FastAPI will then use the `jsonable_encoder` to convert the data to something that can be serialized to JSON and then send it as JSON Lines.

Python 3.10+

```python
# Code above omitted 👆

@app.get("/items/stream-no-annotation")
async def stream_items_no_annotation():
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream")
async def stream_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async")
def stream_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation")
async def stream_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation")
def stream_items_no_async_no_annotation():
    for item in items:
        yield item
```

Server-Sent Events (SSE)

FastAPI also has first-class support for Server-Sent Events (SSE), which are quite similar but with a couple of extra details. You can learn about them in the next chapter: Server-Sent Events (SSE). 🤓